SemLake: Zero-Copy Branching and Three-Way Semantic Merging for Data Lakehouses

Predicate-Level Conflict Verification over Deletion Vectors and Extensions to Agentic Workloads

Xiaofan Cao
Qatar University
Doha, Qatar
xc2314895@qu.edu.qa

ABSTRACT

While modern cloud data lakehouses enable zero-copy branching for isolated data pipelines, their merge mechanics remain fundamentally brittle. Current systems rely on physical file or row-identifier checks, creating a damaging dichotomy: concurrent modifications to disjoint rows in shared files trigger false conflict aborts and severe copy-on-write amplification, whereas cross-file dependencies—such as data deletions concurrent with aggregate calculations—merge blindly, causing silent semantic corruption. Conversely, verified three-way merge formalisms from software version control operate on abstract syntax trees and cannot scale to petabyte-scale cloud storage without prohibitive I/O and API overhead.

This paper presents SemLake, a lakehouse storage and catalog architecture that establishes predicate-level three-way semantic merging over table snapshots without rewriting base columnar files. SemLake formalizes a purge-safe merge invariant that decouples semantic conflict verification from physical data reconciliation, verifying branch compatibility at metadata time with complexity proportional only to branch delta size. Our investigation yields four key insights: (1) predicate-level verification completely eliminates silent semantic corruption across dependent queries, deletes, and updates, capturing subtle integrity violations that escape both file- and row-level engines; (2) decoupling row updates from file boundaries eliminates false write conflicts, achieving near-optimal write amplification during concurrent updates; (3) metadata-driven conflict checking keeps the number of object-store requests per merge independent of table size, about three orders of magnitude fewer than copy-on-write merging; and (4) when extended beyond standard data pipelines to high-concurrency autonomous agent swarms, this semantic merge model resolves severe branch fan-in contention without human intervention. SemLake demonstrates that semantic branch merging can be achieved at cloud scale, bridging the gap between database consistency and zero-copy dataset versioning.

PVLDB Reference Format:
Xiaofan Cao. SemLake: Zero-Copy Branching and Three-Way Semantic Merging for Data Lakehouses. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/WanderingSnail/semlake_milestone2.

1 INTRODUCTION

Cloud data lakehouses [6, 21, 23, 35] decouple compute engines from cloud object storage (such as AWS S3) and expose transactional table snapshots over immutable Parquet files. To isolate data engineering jobs, write-audit-publish (WAP) quality gates, regulatory data scrubs, and collaborative analytics, dataset versioning systems [7, 8, 22, 26, 34] and zero-copy lakehouse catalogs [18, 19, 27, 32] allow users to fork a table snapshot in milliseconds by copying a metadata pointer, execute transformations on an isolated branch, and merge the verified branch back into `main`.

Creating a branch on cloud object storage is cheap because immutable Parquet files are shared across snapshots [8]. Merging two diverged table branches, however, lacks a sound correctness model at lakehouse scale. In production data lakehouses, concurrent branches interact across four fundamental workload patterns:

- W_1 **(Shared-file disjoint updates):** Branches update/delete non-overlapping rows in the same multi-hundred-megabyte Parquet files.
- W_2 **(Hotspot key collisions):** Branches concurrently modify overlapping keys under varying contention density ρ, requiring structured isolation without abort storms.
- W_3 **(Semantic dependencies):** One branch deletes/modifies rows (e.g., regulatory purge) that a concurrent branch read to compute an aggregate or assert an invariant, writing output to a disjoint file.
- W_4 **(Schema evolution and manifest scale):** One branch evolves schema while a concurrent branch appends data across deep manifests and multi-hop DAGs.

State-of-the-art open table formats and versioned databases evaluate conflicts solely at physical file granularity or row-identifier granularity, failing across W_1–W_4. Under file-level optimistic concurrency control (B_1) [6], W_1 triggers false conflict aborts, while copy-on-write row merge (B_2) [32] and tuple-dictionary versioning (B_3) [22, 26] incur severe write amplification or $O(N_{\text{files}})$ S3 requests on W_4. On W_2, existing engines either abort the entire branch (B_1–B_3) or silently overwrite colliding cells via last-writer-wins CRDTs (B_4) [17]. Worse, as shown in Figure 1, all file-, row-, and cell-level engines (B_1–B_4, Dolt [14]) fail on W_3: when a compliance branch T_A purges VIP records in a base file while an analytics branch T_B aggregates revenue over those same records into a new

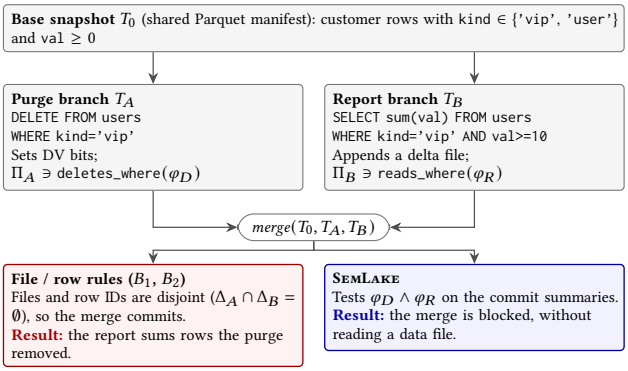


Figure 1: The lakehouse branch-merge dilemma on W_3. A compliance purge branch (T_A) and an analytical report branch (T_B) touch disjoint files and disjoint row identifiers. File- and row-level engines commit both branches silently, whereas SemLake intercepts the predicate conflict ($\varphi_D \wedge \varphi_R$) purely from commit metadata.

summary file, the modified file sets and row IDs are disjoint, causing existing systems to commit a corrupted snapshot.

In the programming-languages (PL) community, three-way structured and semantic merges have been formalized using verification conditions [30], version-space algebras [38], AST matching [37], and post-merge static analysis or unit testing [12, 13]. However, PL techniques operate on abstract syntax trees measured in kilobytes. Lakehouse tables span billions of rows across thousands of cloud objects, where scanning data files at merge time incurs prohibitive S3 API latencies and byte transfer charges [15]. Thus, a lakehouse merge engine must satisfy four core system objectives, which serve as our unified evaluation metrics across W_1–W_4:

- M_1 **(Semantic correctness on W_1–W_4):** 100% recall on cross-branch semantic (W_3) and schema (W_4) conflicts, hotspot collision isolation (W_2), and 0% false aborts on shared files (W_1).
- M_2 **(Sub-second merge latency on W_2, W_4):** Scale three-way diff and commit latency with branch delta size rather than total file count (W_4), DAG depth (W_4), or contention ρ (W_2).
- M_3 **(Cloud cost efficiency on W_1, W_4):** Bound S3 API calls to $O(\Delta_{\text{manifest}})$ and maintain WAF ≈ 1 without full-file CoW rewrites.
- M_4 **(Bounded post-merge read overhead on W_1):** Preserve high-throughput analytical scans over merged snapshots prior to compaction.

To meet M_1–M_4 simultaneously, we present SemLake, a lakehouse storage and catalog architecture built upon three technical pillars. First, for *zero-copy physical reconciliation* ($W_1, W_4 \rightarrow M_3, M_4$), SemLake grounds row updates on positional Roaring deletion vectors (DVs) [5, 28] and incremental delta manifests, compiling concurrent shared-file updates into bitwise bitmap unions and a single delta Parquet append. Second, for *metadata-time semantic and schema verification* ($W_3, W_4 \rightarrow M_1, M_2$), each commit writes a compact JSON summary capturing row operations, schema changes, delete-time pre-images, and SQL predicates extracted from `SELECT` and `DELETE` statements. At merge time, SemLake locates the lowest common ancestor (LCA) via generation-pruned search, replays non-merge commits since the fork, checks static predicate satisfiability in metadata time, and, when the static test cannot rule out an overlap, checks the deleted rows' values at the fork and at delete time. Third, for *structured conflict isolation* ($W_2 \rightarrow M_1, M_2$), true cell collisions are preserved in VSA choice nodes [38] in catalog metadata rather than triggering abort storms.

Finally, beyond standard lakehouse pipelines, we evaluate SemLake on an application extension: high-concurrency autonomous agent swarms (W_{swarm}) [4, 20, 25, 36]. When N AI agents concurrently fork a shared table, execute a composite mix of W_1, W_2, and W_3 operations, and race to advance `main` via catalog compare-and-swap (CAS), SemLake turns lost CAS races into fast server-side semantic merges with group commit, satisfying M_1–M_3 under extreme branch fan-in.

In summary, this paper makes five contributions:

(1) **Unified Three-Way Merge Model:** We formalize four criteria for W_1–W_4—disjoint convergence (W_1), VSA hotspot isolation (W_2), purge-safe merging (W_3), and schema subtyping (W_4)—and show that purge-safety and two-branch serializability (B_6) are incomparable (Section 3).
(2) **Two-Tier Predicate Verifier:** A metadata-time static overlap analyzer paired with a check of the deleted rows' recorded values that clears false conflicts without sacrificing recall (Section 4).
(3) **DAG Replay and Bounds (M_1–M_4):** An exact replay-since-fork algorithm over diamond commit DAGs with Roaring DV reconciliation, backed by formal correctness (M_1), latency (M_2), S3 API cost (M_3), and post-merge scan overhead (M_4) bounds (Sections 4, 5).
(4) **System Implementation:** We implement SemLake over Parquet, Roaring bitmaps, DuckDB, and SQLGlot, with request counting under an S3 price and latency model, and a comparison against the Dolt binary (Section 6).
(5) **Comprehensive Evaluation:** We evaluate SemLake across W_1–W_4, TPC-H refresh pairs, and the W_{swarm} agent extension against B_1–B_6 along metrics M_1–M_4 (Section 7).

2 BACKGROUND AND MOTIVATION

2.1 Lakehouse Storage, Deletion Vectors, and Cloud Costs

Open lakehouse table formats such as Delta Lake [6], Apache Iceberg [5], and Apache Hudi [23, 31] layer transactional metadata over immutable columnar Parquet files on cloud object stores [21, 35]. A table state is defined by an immutable snapshot pointing to a manifest tree of data files. Because S3 objects cannot be modified in place, early lakehouse deployments implemented row-level `UPDATE` and `DELETE` operations via copy-on-write (CoW): modifying a single record inside a 512 MB Parquet file required reading the entire file and writing a new 512 MB copy [10], resulting in extreme write amplification (WAF $\gg 1$).

As Durner et al. [15] demonstrate, cloud object storage performance and billing for metadata-intensive operations are governed by API request latencies (30 ms to 100 ms per call) and per-request pricing ($0.005 per 1,000 `PUT`/`LIST` calls and $0.0004 per 1,000 `GET` calls) rather than raw bandwidth. To avoid full-file rewrites, Okolnychyi et al. [28], the Apache Iceberg v3 specification [5], and Delta Lake deletion vectors [24] adopt merge-on-read (MoR) with positional deletion vectors (DVs). A DV is a compressed Roaring bitmap associated with a base Parquet file that marks the zero-based

positions of deleted or superseded rows. An update sets the old row position in the base file's DV and appends the updated record to a small delta Parquet file. Analytical readers apply the bitmap mask during scans, and background compaction periodically folds DVs back into clean base files. Iceberg v3 further assigns stable row lineage identifiers so logical row identity is preserved across updates.

2.2 Problem Taxonomy (W_1–W_4) and Design Metrics (M_1–M_4)

While prior work benchmarked single-branch table operations in LST-Bench [10], multi-branch development introduces the *Branch-Execute-Merge* lifecycle: $T_0 \xrightarrow{\text{Branch}} (T_A, T_B) \xrightarrow{\text{Execute}} (T'_A, T'_B) \xrightarrow{\text{Merge}} T_M$. When two diverged branches T_A and T_B merge relative to their lowest common ancestor T_0, four canonical workload interaction classes arise:

(1) **Shared-File Disjoint Updates (W_1):** Concurrent branches modify distinct row IDs ($W(A) \cap W(B) = \emptyset$) or disjoint columns co-residing in shared Parquet files, requiring 0% false aborts (M_1), WAF ≈ 1 (M_3), and $< 8\%$ localized scan overhead (M_4).
(2) **Hotspot Key Collisions (W_2):** Concurrent branches update overlapping keys with contention density $\rho \in (0, 1]$, requiring cell collision isolation (M_1) without whole-branch abort storms (M_2).
(3) **Semantic Dependencies (W_3):** Branch A purges rows satisfying φ_D while Branch B reads rows satisfying φ_R to materialize a summary or assert an invariant ($M_1 : \Delta_A \not\models \Pi_B$).
(4) **Schema Evolution and Manifest Scale (W_4):** Branch A evolves schema $S_0 \to S_A$ while Branch B appends data files across N_{files} base files and multi-hop DAGs (M_1, M_2, M_3).

These four workloads directly ground our four evaluation metrics:

- M_1 **(Semantic Correctness on W_1–W_4):** Recall and precision on W_3, W_4 [12, 30], cell collision isolation on W_2, and false-abort rate (FAR) on W_1.
- M_2 **(Merge Latency on W_2, W_4):** Wall-clock three-way diff and commit latency across manifest depth (W_4), DAG history depth (W_4), and contention density ρ (W_2) [23, 27].
- M_3 **(Cloud Cost Efficiency on W_1, W_4):** S3 API requests, dollar cost under the Durner et al. model [15], and write amplification factor (WAF).
- M_4 **(Post-Merge Scan Penalty on W_1):** Relative query latency overhead on merged snapshots T_M before compaction in DuckDB TPC-H scans [10, 33].

2.3 Why Existing Merge Mechanisms Fail (B_1–B_6 and B_1R)

Prior art brackets the lakehouse merge problem from two directions: dataset versioning systems [7, 8, 22, 26, 32, 34] provide scalable branching without semantic merge verification, whereas program-merge verifiers [12, 13, 30, 37, 38] provide three-way guarantees over kilobyte-scale ASTs that cannot scan billion-row S3 tables.

To establish a unified baseline taxonomy across the paper, we categorize existing mechanisms into six baseline families (B_1–B_6, plus B_1R as the row-level write variant of B_1): B_1–B_4 and B_1R represent lakehouse and dataset-versioning architectures, B_5 (Dolt [14]) represents an external constraint-aware cell-merge engine, and B_6 (Serializable Snapshot Isolation, SSI [9]) represents classical OLTP concurrency control. Table 1 summarizes their behavior across W_1–W_4 and M_1–M_4:

- B_1**: Native file-level OCC (Iceberg/Delta default)** [6] intersects modified file paths since T_0, causing 100% false aborts on shared-file updates (W_1), abort storms on W_2, $O(N_{\text{files}})$ manifest scans on W_4, and 0% recall on semantic dependencies (W_3).
- B_1**R: Delta row-concurrency variant of** B_1 [24] resolves W_1 writes via DVs but tracks reads at file granularity (`ConcurrentDeleteReadException`), false-blocking queries when any row in a scanned file is deleted (W_3).
- B_2**: Copy-on-Write row merge (Bauplan/lakeFS)** [32] resolves row IDs on W_1 with 0% read penalty (M_4), but rewrites entire files on every merge (M_3 : WAF $\gg 1$), aborts whole branches on W_2, and scores 0% recall on W_3.
- B_3**: Relational tuple versioning (Decibel/OrpheusDB)** [22, 26] tracks per-tuple bitmaps in local DBMS storage; rebuilding per-file dictionaries on cloud lakehouses incurs $O(N_{\text{files}})$ S3 calls (M_2, M_3) and misses read predicates on W_3.
- B_4**: Automatic CRDT views (CRDV)** [2, 17] guarantees state convergence via last-writer-wins, overwriting updates on W_2 and admitting semantic violations on W_3 without audit gates.
- B_5**: Constraint-aware cell merge (Dolt)** [14] merges Prolly trees at cell granularity and validates declared `FOREIGN KEY` and `CHECK` constraints, but misses query read predicates (W_3) and rejects `ALTER TABLE ADD CONSTRAINT CHECK` when legacy T_0 rows violate the new bound.
- B_6**: Classical Serializable Snapshot Isolation (SSI)** [1, 9, 16] tracks antidependencies in volatile lock tables for online OLTP, permitting the one-way schedule $T_B \prec T_A$ which violates purge-safety when merged into a single published snapshot T_M (Proposition 3.6).

2.4 Extension Beyond Lakehouse Pipelines: Concurrent Agent Workloads (W_{swarm})

While three-way semantic merge is primarily designed for lakehouse ETL, backfill, and WAP pipelines (W_1–W_4), emerging autonomous agent workflows (e.g., Git4Data [20], Chronos [36], BranchBench [4], ATR [25]) extend database branching to multi-agent swarms. Under high concurrency, git-style data branching faces severe merge bottlenecks. We define W_{swarm} as a composite fan-in workload where $N \in [8, 64]$ autonomous agents concurrently fork from `main`, execute mixed SQL operations combining W_1 (disjoint row updates), W_2 (hotspot key updates), and W_3 (range deletes vs. aggregate queries), and race to publish via catalog CAS. Under B_1 or B_1R, losing agents suffer abort storms or false read blocks. Extending SemLake with a server-side CAS merge service and group commit turns lost CAS races into metadata-time semantic merges, optimizing M_1 (zero committed hazards), M_2 (high throughput), and M_3 (low WAF) under extreme fan-in.

Table 1: Architectural comparison of existing merge and concurrency paradigms (B_1–B_6 and B_1R) against SemLake across all four lakehouse workload classes (W_1–W_4) and evaluation metrics (M_1–M_4).

System / Baseline	Granularity	W_1: Shared-File Updates (M_1, M_3, M_4)	W_2: Hotspot Key Collisions (M_1, M_2)	W_3: Semantic Dependencies (M_1)	W_4: Schema & Manifest Scale (M_1, M_2, M_3)
B_1: File-level OCC [6]	Physical file	Aborts (FAR = 100% on shared files)	Whole-branch file-conflict abort storm	Commits silently (0% recall)	$O(N_{\text{files}})$ manifest scan; no 3-way schema unify
B_1R: Delta row-write variant of B_1 [24]	Row write DV + file read	Commits via DVs (FAR = 0%, low WAF)	Row-conflict abort & full transaction retry	False-blocks all readers of touched file	$O(N_{\text{files}})$ manifest scan on merge
B_2: Copy-on-Write row merge [32]	Row ID (CoW)	Full file rewrite (WAF $\gg$ 1, 0% scan pen.)	Whole-branch abort & full file rewrite	Commits silently (0% recall)	O(TableBytes) full file rewrite I/O
B_3: Relational tuple versioning [22, 26]	Tuple version dictionary	Rebuilds tuple-version records for every touched file	Whole-branch primary-key conflict abort	Commits silently (0% recall on reads/domains)	$O(N_{\text{files}})$ S3 API calls to rebuild tuple dictionaries
B_4: Automatic CRDT views (CRDV) [17]	Operation log	Commits via log	Silent LWW overwrite (loses updates)	Commits silently (0% recall)	$O(\lvert\text{Log}\rvert)$ replay; static view schema
B_5: Constraint-aware cell merge (Dolt) [14]	Prolly-tree cell + SQL constraint	Commits via cell diff	Halts merge in dirty cell-conflict state	Misses query reads (recall 0.11; FK/CHECK only)	$O(\log N_{\text{cells}})$ tree diff; rejects legacy-violating CHECK
B_6: Serializable Snapshot Isolation (SSI) [9]	Online OLTP SIREAD lock	Commits if tuple locks disjoint	Interactive transaction abort on ww-conflict	Permits 1-way $T_B \prec T_A$ (violates purge-safety)	Volatile lock table; no branched S3 manifests
SemLake (Ours)	**Row DV + SQL Predicate + VSA**	**Roaring DV union (FAR = 0%, no file rewrite, ++0.9% localized scan overhead)**	**Composes disjoint cols; isolates cell collisions in VSA ($\mathcal{V}_M$)**	**Two-tier predicate & pre-image check (100% recall)**	**Bidirectional schema unify + $O(\Delta_{\text{manifest}})$ delta manifest (7 S3 calls)**

3 FORMAL DATA MODEL AND THREE-WAY MERGE INVARIANTS

We now formalize the lakehouse snapshot model, branch deltas, and the four unified three-way merge criteria that govern W_1–W_4 in one-to-one correspondence.

3.1 Snapshot DAG, Commit Summaries, and Branch Deltas

Let $\mathcal{R}$ be a universe of stable row identifiers (matching Iceberg v3 row lineage IDs [5]) and $\mathcal{A}$ be a universe of column names.

Definition 3.1 (Table Snapshot). A table snapshot is a tuple $T = (\text{id}, P, S, \rho, \Phi)$, where id is a unique snapshot identifier, $P \subseteq \text{id}^*$ is a tuple of parent snapshot identifiers ($|P| \in \{0, 1, 2\}$), $S = (a_1 : \tau_1, \ldots, a_k : \tau_k)$ is the table schema, $\rho : \mathcal{R} \rightharpoonup (\mathcal{A} \rightharpoonup \mathcal{V})$ is a partial map from live row IDs to column-value records, and Φ is a map from file identifiers to physical file descriptors (k_{obj}, size, token, DV).

A snapshot with $|P| = 1$ is a regular commit; a snapshot with $|P| = 2$ is a merge commit. The snapshots of a repository form a directed acyclic graph (DAG) $G = (V, E)$ with edges (u, p) for each $p \in P(u)$. Each snapshot u has a generation number $\text{gen}(u)$, defined as 0 if $P(u) = \emptyset$ and $1 + \max_{p \in P(u)} \text{gen}(p)$ otherwise. Let $\text{anc}(u)$ denote the set of all ancestors of u (including u itself).

Definition 3.2 (Fork Point). Given two branch tips T_A and T_B, their fork point is the shared ancestor of maximum generation: $T_0 = \text{LCA}(T_A, T_B) = \arg\max_{u \in \text{anc}(T_A) \cap \text{anc}(T_B)} \text{gen}(u)$, breaking ties deterministically by snapshot identifier.

Definition 3.3 (Commit Summary and Folded Branch Delta). Each non-merge commit c records a delta summary $\delta(c) = (\text{ops}_c, S_c, \Pi_c, \sigma_c)$, where $\text{ops}_c : \mathcal{R} \rightharpoonup \{\texttt{insert}, \texttt{update}, \texttt{delete}\} \times (\mathcal{A} \rightharpoonup \mathcal{V})$ maps modified row IDs to operation kinds and column values; S_c is the schema; Π_c is a tuple of attached semantic predicates; and $\sigma_c : \mathcal{R} \rightharpoonup (\mathcal{A} \rightharpoonup \mathcal{V})$ maps deleted rows to their pre-images immediately before deletion. For branch tip T_B and fork T_0, let $C(T_0, T_B) = \{c \in \text{anc}(T_B) \setminus \text{anc}(T_0)\}$. Folding $C(T_0, T_B)$ in topological order yields the folded branch delta $\Delta_B = (W(B), D(B), S_B, \Pi(B))$, comprising write set $W(B) = (\text{ops}_B, \sigma_B)$, deleted row set $D(B) = \{r \mid \text{ops}_B(r) = (\texttt{delete}, \cdot)\}$, tip schema S_B, and predicates $\Pi(B)$.

3.2 Unified Three-Way Merge Criteria Across W_1–W_4

A valid merged snapshot $T_M = \text{merge}(T_0, T_A, T_B)$ must simultaneously satisfy four formal criteria corresponding one-to-one with W_1–W_4:

Criterion 1 (W_1: Disjoint Row and Column Convergence). For each row ID $r \in \mathcal{R}$ modified in $\text{ops}_A \cup \text{ops}_B$: if r is modified by only one branch $X \in \{A, B\}$ or modified identically by both ($\text{ops}_A(r) = \text{ops}_B(r)$), $\rho_M(r)$ adopts $\text{ops}_X(r)$ directly; if both branches update r on disjoint column subsets ($\text{cols}_A(r) \cap \text{cols}_B(r) = \emptyset$), the updates commute and converge into $\rho_M(r) = \rho_0(r) \oplus \Delta_A(r) \oplus \Delta_B(r)$ regardless of physical file co-location.

Criterion 2 (W_2: Hotspot Cell-Collision Isolation via VSA).. If both branches assign distinct values $v_A \neq v_B$ to the same column $c \in \text{cols}_A(r) \cap \text{cols}_B(r)$ or one updates r while the other deletes r, the conflicting cell is encapsulated without data loss into a Version Space Algebra (VSA) [38] choice node $\texttt{choice}(r, c, [(A, v_A), (B, v_B)]) \in \mathcal{V}_M$, while non-conflicting columns of r still converge in $\rho_M(r)$.

Criterion 3 (W_3: Cross-Branch Predicate Preservation). Table 2 defines the five predicate kinds in SemLake: $\texttt{reads_where}(\varphi_R)$ and $\texttt{deletes_where}(\varphi_D)$ capture the selection scope of SQL SELECT and DELETE statements; $\texttt{rows_live}(I)$ and $\texttt{foreign_key}(I)$ specify finite row-ID sets $I \subset \mathcal{R}$ that must remain live in T_M; and $\texttt{domain}(\varphi_{\text{dom}})$ asserts a value invariant over concurrent writes ops_X. Unlike a table-wide SQL CHECK constraint in B_5 (Dolt), a domain predicate validates only the new assignments in ops_X since T_0 and does not fail if untouched legacy records in T_0 lie outside φ_{dom}.

Criterion 4 (W_4: Bidirectional Schema Subtyping Compatibility). Let $\preceq_{\text{type}}$ denote the lossless numeric widening preorder ($\texttt{int32} \prec_{\text{type}} \texttt{int64} \prec_{\text{type}} \texttt{double}$, and $\tau \preceq_{\text{type}} \tau$ for all types τ). Tip schemas S_A and S_B are merge-compatible relative to S_0 iff for every column $a \in \mathcal{A}$: (a) neither branch drops or renames a column present in S_0; (b) any column $a \notin S_0$ added by S_A or S_B is nullable (and if added by both, has a common supertype); and (c) for every $a \in S_0$, $\tau_0(a) \preceq_{\text{type}} \tau_A(a)$ and $\tau_0(a) \preceq_{\text{type}} \tau_B(a)$, yielding the unified schema $S_M(a) = \sup_{\preceq_{\text{type}}}(\tau_A(a), \tau_B(a))$ without rewriting base Parquet files.

Table 2: Taxonomy of predicate kinds supported by SemLake for workload W_3, their SQL or programmatic origin, and the formal three-way conflict rule evaluated between branch deltas Δ_A and Δ_B.

Predicate Kind	Attached By	Representative Example	Three-Way Conflict Rule (Δ_A vs. Π_B)
`reads_where`(φ_R)	Reader / aggregate commit (W_3)	`kind = 'vip' AND value >= 10`	Conflicts with `deletes_where`(φ_D) $\in \Pi(A)$ if Overlap(φ_D, φ_R) holds and the deleted-row check does not clear it, and with an update or insert in ops_A that moves a row across φ_R or changes a column the reader uses on a row inside it.
`deletes_where`(φ_D)	Delete / purge commit (W_3)	`l_shipdate < 19930201`	Declares the selection scope of a delete operation; paired symmetrically against every `reads_where`(φ_R) $\in \Pi(B)$.
`rows_live`(I)	Point-read / lineage commit (W_3)	$I = \{101, 102, 105\}$	Conflicts if the partner branch deletes any listed row identifier: $D(A) \cap I \neq \emptyset$.
`foreign_key`(I)	Child-row insert commit (W_3)	$I = \{1\}$ (`orders.user_id` → `users.id`)	Conflicts if the partner branch deletes any referenced parent row identifier: $D(A) \cap I \neq \emptyset$.
`domain`(φ_{dom})	Invariant / policy commit (W_3)	`value >= 0`	Conflicts if any `insert` or `update` in ops_A assigns column values that violate φ_{dom}.

Definition 3.4 (Three-Way Table Merge Operator). A three-way merge operator merge(T_0, T_A, T_B) evaluates Criteria 1–4 across W_1–W_4: (1) If Criteria 3 and 4 hold, it returns merged snapshot T_M with parents (T_A, T_B), converged row state ρ_M (W_1, status `clean`, outcome `keep` when $\mathcal{V}_M = \emptyset$), VSA choices $\mathcal{V}_M$ (W_2, status `vsa`, outcome `replan` when $\mathcal{V}_M \neq \emptyset$), and unified schema S_M (W_4); (2) `semantic_conflict`(V) (`block`) if predicate violations $V \neq \emptyset$ violate Criterion 3 (W_3); or (3) `schema_conflict` (`block`) if schemas violate Criterion 4 (W_4).

3.3 Purge-Safe Merge Invariant vs. Classical Serializability (W_3 vs. B_6)

We now formalize Criterion 3 (W_3) as the *purge-safe merge invariant* and prove its relationship to classical transaction serializability (B_6, SSI) [1, 9, 16].

Definition 3.5 (Purge-Safe Merge). Let T_A, T_B be branches with fork T_0. A merge $T_M = \text{merge}(T_0, T_A, T_B)$ is *purge-safe* if, symmetrically for $(X, Y) \in \{(A, B), (B, A)\}$: (1) for every `reads_where`($\varphi_R$) $\in \Pi(Y)$ and every row $r \in D(X)$, neither the row's value at the fork nor its pre-image at delete time satisfies φ_R: $\rho_0(r) \not\models \varphi_R$ (when r exists in T_0) and $\sigma_X(r) \not\models \varphi_R$; (2) for every `rows_live`($I$) or `foreign_key`($I$) $\in \Pi(Y)$, $D(X) \cap I = \emptyset$; (3) for every `domain`(φ_{dom}) $\in \Pi(Y)$ and $(r, v) \in \text{ops}_X$, $v \models \varphi_{\text{dom}}$; and (4) for every `reads_where`(φ_R) $\in \Pi(Y)$ and every row r that X updates or inserts and does not delete, with value $\rho_X(r)$ at X's tip, r stays on the same side of the clause, $\rho_0(r) \models \varphi_R \Leftrightarrow \rho_X(r) \models \varphi_R$ (an inserted row is outside at the fork), and if it is inside, $\rho_0(r)$ and $\rho_X(r)$ agree on $\text{cols}(\varphi_R)$, the columns the reader uses outside its `WHERE` clause.

Proposition 3.6 (Purge-Safety vs. Serializability (B_6)). *Purge-safety and two-branch conflict serializability are incomparable. There exist branch pairs (T_A, T_B) that admit a serializable execution order $T_B \prec T_A$ under SSI (B_6) [9] yet violate purge-safety, and purge-safe merges that are not serializable.*

Proof. Consider Figure 1. Branch T_B reads rows satisfying φ_R = (`kind='vip'` $\wedge$ `val` ≥ 10) from T_0 and inserts a summary row into a new file. Branch T_A deletes rows satisfying φ_D = (`kind='vip'`) from T_0 and reads nothing from T_B. In Adya's dependency graph [1], there is a single antidependency $T_B \xrightarrow{rw} T_A$ and no edge $T_A \to T_B$. Because a cycle requires two consecutive *rw*-edges [9], the graph is acyclic and the serial schedule $T_B \prec T_A$ is accepted by SSI. However, in T_M, both the erasure of VIP records and the derived summary coexist in the same published snapshot. Because the deleted VIP rows satisfy φ_R, Definition 3.5 rejects the merge. Conversely, let both branches update the same cell of the same row to different values, with no predicates. Definition 3.5 holds trivially, but any serial order loses one of the two writes, so a merge that keeps both outcomes as a VSA choice (Criterion 2) is not equivalent to a serial execution. Hence neither property implies the other. □

In a branched lakehouse, T_M is published as a single coherent dataset snapshot: if T_A is a GDPR erasure or a scrub of corrupted records, admitting T_B's pre-purge aggregate means T_M publishes statistics derived from records it claims were erased. When an application instead desires directional serializability (accepting $T_B \prec T_A$), SemLake supports an optional directional mode; by default, it enforces symmetric purge-safety.

3.4 Decidable SQL Predicate Fragment ($\mathcal{F}$)

To evaluate $\varphi_D \wedge \varphi_R$ in metadata time without touching data files, SemLake defines a static conjunctive fragment $\mathcal{F}$ over column names $c \in \mathcal{A}$ and literals ℓ: $\varphi ::= t_1 \wedge \cdots \wedge t_m$, where each atomic constraint t_i takes one of three forms: (1) binary comparisons c op ℓ with op $\in \{=, \neq, <>, <, \leq, >, \geq\}$; (2) finite set membership c `IN` $(\ell_1, \ldots, \ell_k)$; or (3) closed intervals c `BETWEEN` ℓ_{low} `AND` ℓ_{high}. Clauses containing `OR`, `NOT`, arithmetic expressions, joins, or subqueries lie outside $\mathcal{F}$ and are conservatively routed to the delete-time row-value post-check (Section 4.4).

4 SYSTEM ARCHITECTURE AND MERGE DESIGN

4.1 Architectural Overview

Figure 2 illustrates the architecture of SemLake. The system layers metadata-driven branch management and three-way semantic merge execution over an immutable cloud object store across four decoupled components:

(1) **Catalog and DAG Layer (M_2):** Maintains branch head pointers (`heads`$[b] \to$ id) updated via atomic compare-and-swap (CAS), tracks the snapshot parent DAG, and caches ancestor-generation maps for sub-millisecond LCA discovery (W_4, W_{swarm}).
(2) **Commit Summary and SQL Predicate Layer (M_1, M_3):** Extracts read and delete predicates from SQL ASTs (W_3) and writes a compact commit summary (`manifests/<id>.summary.json`) with row deltas, schemas, predicates, and delete-time pre-images.
(3) **Three-Way Semantic Merge and VSA Engine (M_1, M_2):** Given tips (T_A, T_B) and fork point T_0, replays non-merge commit summaries since T_0, executes the two-tier predicate verifier (W_3), unifies schemas (W_4), and reconciles row updates into converged records (W_1) or VSA choice nodes (W_2).

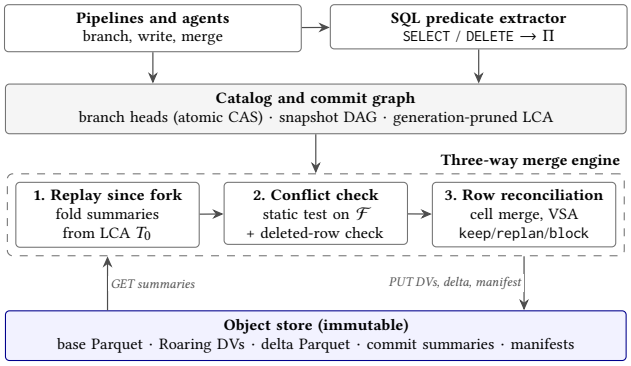


Figure 2: System architecture of SemLake. Merging two diverged branches reads only the compact commit summaries since the fork point T_0 and publishes a single delta Parquet file, updated Roaring deletion vectors, and a delta manifest list.

(4) **Deletion-Vector Storage Substrate (M_3, M_4):** Keeps base Parquet files immutable, encodes deleted and superseded row positions into serialized Roaring bitmaps [28], writes updates into a single delta Parquet file, and publishes a two-element delta manifest list at $O(\Delta_{\text{manifest}})$ S3 API cost (W_1, W_4).

4.2 Compact Commit Summaries and SQL Extraction

In standard Iceberg or Delta tables, discovering which rows changed between two snapshots requires reading manifest trees and scanning data files or change-data-capture logs. SemLake eliminates data scans during merge by writing one compact JSON summary per commit containing: (1) parent snapshot IDs, branch name, and schema fields (W_4); (2) modified file IDs and their updated descriptors, including base64-encoded Roaring DVs (W_1); (3) row-level operations mapping each modified row ID to its operation kind, column assignments, and pre-image values upon deletion (W_1, W_2, W_3); (4) attached predicates (W_3); and (5) any open VSA choice nodes (W_2). Replaying a non-merge commit during a three-way merge issues exactly one object-store `GET` request for its summary object, regardless of whether the table contains 10^3 or 10^5 unchanged Parquet files.

Writers can attach predicates programmatically or pass raw SQL statements to SemLake's SQL extractor. The extractor parses `SELECT` and `DELETE` statements into an AST, normalizes the `WHERE` clause into canonical SQL text, and emits `reads_where`(φ_R) or `deletes_where`(φ_D). Queries containing joins, `UNION`, or subqueries retain their full clause text so the conflict verifier conservatively classifies them as outside $\mathcal{F}$ and falls back to the deleted-row check.

4.3 DAG-Aware Delta Replay Since the Fork

When long-running branches merge repeatedly across multi-stage pipelines (W_4), the commit history forms a DAG with nested merge diamonds rather than a simple tree. A naive three-way merge that inspects only the tip summary of each parent fails in two ways: first, a merge commit's summary does not duplicate ancestor predicates or rows parked in VSA choices, so a subsequent merge would forget earlier read or delete constraints (W_3); second, walking all ancestors

Algorithm 1: DAG-Aware Replay Since the Fork (`ReplaySinceFork`)

Input: Repository *repo*, object store *store*, fork ID b, branch tip ID t
Output: Folded branch delta $\Delta_t = (ops, changed, S, \Pi, \Phi, vsa, \sigma)$
1: **if** $t = b$ **then return** EmptyDelta(S_b)
2: *contained* ← *repo*.ancestors(b) ▷ Cached ancestor set of fork b
3: *order* ← TopologicalWalk(t, skip = *contained*) ▷ Post-order DFS
4: *folded*[b] ← EmptyDelta(S_b)
5: **for each** $u \in$ *order* **do**
6: *snap* ← *repo*.snapshots[u]
7: **if** |*snap*.parents| ≥ 2 **then** ▷ Internal merge diamond
8: (p_1, p_2) ← *snap*.parents; f ← LCA(p_1, p_2)
9: **if** $f \notin$ *folded* **then** $f \leftarrow b$
10: L ← StripPrefix(Known(p_1), *folded*[f])
11: R ← StripPrefix(Known(p_2), *folded*[f]); br ← Combine(L, R, S_b)
12: *folded*[u] ← FoldCommit(*folded*[f], br, ρ_b)
13: **else** ▷ Regular commit: 1 summary `GET`
14: p ← *snap*.parents[0] **if** *snap*.parents **else** b
15: *step* ← ParseSummary(*store*.get(*snap*.summary_key))
16: *folded*[u] ← FoldCommit(Known(p), *step*, ρ_b)
17: **return** *folded*[t]

Algorithm 2: Two-Tier Semantic Conflict Check (`VerifyMergeSemantics`)

Input: Folded branch deltas Δ_A, Δ_B, fork rows ρ_0
Output: Violation list V ($V = \emptyset$ iff merge is purge-safe)
1: $V \leftarrow [\,]$; $D_X, U_X \leftarrow$ deleted, changed IDs in Δ_X.ops, $X \in \{A, B\}$
2: **for each** $(X, Y) \in \{(A, B), (B, A)\}$ **do**
3: $moved_X \leftarrow \{r \in D_X \mid \sigma_X(r) \neq \rho_0(r)\}$ ▷ Rows updated before deletion
4: **for each** $p \in \Delta_Y.\Pi$ **with** p.kind ∈ {`rows_live`, `foreign_key`} **do**
5: **if** p.row_ids ∩ $D_X \neq \emptyset$ **then** Append(V, Violation(p, p.row_ids ∩ D_X))
6: **for each** $s \in \Delta_X.\Pi$ **with** s.kind = `deletes_where` **do**
7: **for each** $p \in \Delta_Y.\Pi$ **with** p.kind = `reads_where` **do**
8: **if not** WhereOverlap($s.\varphi_D, p.\varphi_R$) **then** $R \leftarrow moved_X$; *vals* ← $\{\rho_0\}$
9: **else** $R \leftarrow D_X$; *vals* ← $\{\rho_0, \sigma_X\}$
10: **if** AnyMatch($R, p.\varphi_R$, *vals*) **then** Append(V, Violation(p, s))
11: **for each** $p \in \Delta_Y.\Pi$ **with** p.kind = `domain` **do**
12: *bad* ← $\{r \mid (r, (op, vals, \cdot)) \in \Delta_X.\text{ops},\ op \in \{\texttt{insert}, \texttt{update}\},\ vals \not\models p.\varphi_{\text{dom}}\}$
13: **if** *bad* ≠ ∅ **then** Append(V, Violation(p, *bad*))
14: **for each** `reads_where` $p \in \Delta_Y.\Pi$ **with** $M \leftarrow \{r \in U_X \mid \text{Unstable}(r, p)\} \neq \emptyset$ **do** Append(V, Violation(p, M))
15: **return** V

of an internal merge commit without stripping commits already present in the new fork point $b = \text{LCA}(T_A, T_B)$ would either replay shared history prefixes as false conflicts (W_1, W_2) or drop commits whose parent predates b.

Algorithm 1 (`ReplaySinceFork`) reconstructs the exact folded delta Δ_t of branch tip t since fork point b. First, it retrieves the cached ancestor set *contained* = anc(b) and executes a post-order depth-first walk from t that halts upon entering *contained*. Every commit in anc(t) \ anc(b) is emitted once in topological order (parents before children), mapping any parent inside anc(b) to *folded*[b] via Known(p).

Second, for each regular commit u, Algorithm 1 fetches u's summary object (1 `GET`) and folds it onto its parent's delta via `FoldCommit`. If u updates a row r previously parked in an open VSA choice, `FoldCommit` resolves the VSA choice for r and records the new write; if u deletes r, it records r's delete-time pre-image in $\sigma[r]$. Third, when the walk encounters an internal merge node u with parents (p_1, p_2), it locates u's internal fork $f = \text{LCA}(p_1, p_2)$, strips the shared prefix *folded*[f] from both parent deltas via `StripPrefix`, and combines their relative deltas without extra object-store reads.

4.4 Two-Tier Semantic Conflict Verification (W_3)

Given branch deltas Δ_A, Δ_B, Algorithm 2 (`VerifyMergeSemantics`) evaluates Criterion 3 (Definition 3.5) symmetrically for $(X, Y) \in \{(A, B), (B, A)\}$ across four verification stages:

1. Tier-1 Static Satisfiability (`WhereOverlap`). For each pair of `deletes_where`(φ_D) $\in \Delta_X.\Pi$ and `reads_where`(φ_R) $\in \Delta_Y.\Pi$, `WhereOverlap` tests whether $\varphi_D \wedge \varphi_R$ is satisfiable. When both clauses parse into the conjunctive fragment $\mathcal{F}$, atomic constraints are

grouped by column c. For each column c, the solver intersects the = and IN candidates, removes ≠ points, and keeps the tightest lower and upper bounds with their strictness; c is unsatisfiable iff the interval is empty or excludes every remaining candidate. If any column is unsatisfiable, $\varphi_D \wedge \varphi_R$ is unsatisfiable and `Where Overlap` returns `False` in microseconds without reading a single table row. No row deleted under φ_D can then satisfy φ_R with its delete-time values; the one remaining case is a row that X updated before deleting it, whose fork value (which Y read) may still satisfy φ_R, so Line 8 checks the fork values of exactly those rows.

2. Tier-2 Deleted-Row Value Check (`AnyMatch`). When the static test cannot rule out an overlap, either because a clause falls outside $\mathcal{F}$ (e.g., `DELETE WHERE NOT kind = 'vip'`) or because the two clauses overlap on a region holding no deleted row, Lines 9–10 evaluate φ_R on each deleted row's value at the fork and at delete time. Both values come from the commit summaries, so no data file is read. Clauses in $\mathcal{F}$ are evaluated directly, and any other single-table clause is evaluated by DuckDB [29] over those few rows. A row with no recorded value, or a read clause with a join or subquery that cannot be evaluated on its own, keeps the conflict. The check costs $O(|D_X|)$ per clause pair.

3. Lineage, Foreign-Key, and Domain Verification. Lines 4–5 intersect each `rows_live` and `foreign_key` set $I \in \Delta_Y.\Pi$ against D_X in $O(\min(|I|, |D_X|))$ time. Lines 11–13 evaluate each $\texttt{domain}(\varphi_{\text{dom}}) \in \Delta_Y.\Pi$ against all `insert` and `update` assignments in Δ_X.ops, flagging any record whose assigned columns violate φ_{dom}.

4. Read Stability under Updates and Inserts. Line 14 check Condition 4 on each updated or inserted row's fork and tip values from the summaries, with the evaluator of `AnyMatch`: `Unstable` reports a row whose membership in φ_R differs, or a row inside on both sides whose changed columns meet $\text{cols}(\varphi_R)$, taken from the `SELECT` outside its `WHERE` clause (`count(*)` uses none; `SELECT *` or a join uses all). A changed row on which φ_R cannot be evaluated counts as crossing. The check costs $O(|U_X|)$ per read predicate.

4.5 Unified State Reconciliation Across W_1–W_4

Combining Algorithm 2 with Criteria 1–4 yields a unified decision and state-reconciliation procedure across W_1–W_4 (where at runtime, the W_3 and W_4 `block` checks execute first as fail-fast metadata guards prior to materializing W_1 and W_2 row deltas):

(1) **Disjoint row/column convergence ($W_1 \rightarrow$ `keep`):** Under Criterion 1, rows modified by only one branch or identically on both adopt $\text{ops}_X(r)$. Rows updated on disjoint columns commute into $\rho_0(r) \oplus \Delta_A(r) \oplus \Delta_B(r)$ with zero false aborts (M_1).

(2) **Hotspot collision isolation in VSA ($W_2 \rightarrow$ `replan`):** Under Criterion 2, if both branches assign distinct values $v_A \neq v_B$ to column c (or update vs. delete), SemLake constructs a VSA choice node [38] $\texttt{choice}(r, c, [(A, v_A), (B, v_B)]) \in \mathcal{V}_M$. Non-conflicting columns merge cleanly, while the cell retains its base value in the snapshot until resolved (M_1, M_2).

(3) **Semantic conflict blocking ($W_3 \rightarrow$ `block`):** Under Criterion 3, if Algorithm 2 returns $V \neq \emptyset$, the merge halts immediately with `semantic_conflict`(V) before writing data, guaranteeing zero silent corruption (M_1).

(4) **Bidirectional schema unification ($W_4 \rightarrow$ `keep`/`block`):** Under Criterion 4, additive nullable columns and numeric widenings (`int32` → `int64` → `double`) unify into S_M without rewriting base files; incompatible column drops, renames, or narrowings halt with `schema_conflict` (`block`).

4.6 Zero-Copy Physical Merge over Deletion Vectors (M_3, M_4)

SemLake materializes the merged snapshot T_M without rewriting any base Parquet file (M_3, M_4). For every row r updated or deleted in ops_M that resided in a base file f at zero-based offset $\text{pos}_0[r]$, SemLake sets bit $\text{pos}_0[r]$ in f's Roaring deletion vector ($\text{DV}_M(f) = \text{DV}_A(f) \cup \text{DV}_B(f) \cup \{\text{pos}_0[r]\}$). All newly inserted records and resolved row updates across both branches are serialized into a single compact delta Parquet file (`data/<id>/delta-<id>.parquet`).

The commit protocol then writes three small metadata objects: (1) the merge summary `manifests/<id>.summary.json`, (2) a delta manifest `manifests/<id>.delta.json` containing descriptors only for files whose DV or payload changed since T_0, and (3) a two-element manifest list `[base_manifest_key,delta_key]`. Because untouched base file descriptors are inherited by reference, S3 API requests scale as $O(\Delta_{\text{manifest}})$ (M_3). During subsequent analytical queries (M_4), the scan engine inspects each row group's DV: clean row groups ($\text{DV} = \emptyset$) execute native vectorized Parquet scans without filtering overhead, while background compaction periodically folds DVs back into base files once the dirty-row ratio or scan-to-merge ratio crosses the cost threshold.

4.7 Extension: Commit Service for Agent Swarms (W_{swarm})

To support the composite agent-swarm workload W_{swarm} beyond standard WAP pipelines, SemLake provides a server-side CAS merge service. When k agents concurrently invoke CAS(`main`, $T_{\text{expected}}, T_{\text{local}}$), the $k-1$ losing branches are not aborted; instead, the server executes a three-way semantic merge between each losing snapshot and the advancing head (M_1), and returns a commit that conflicts with the head to its agent. With one CAS per commit, throughput is bounded by each merge's catalog and object-store round trips (M_2). With *group commit*, the service takes up to G queued commits, merges them one by one onto the head in memory, and publishes the batch with a single catalog CAS. In this paper the service runs inside the swarm simulation of Section 7.7.

5 CORRECTNESS, COMPLEXITY, AND SCAN OVERHEAD ANALYSIS

We now establish formal guarantees for SemLake across all four evaluation metrics: semantic correctness (M_1), merge latency complexity (M_2), cloud S3 API and write amplification bounds (M_3), and post-merge analytical scan overhead (M_4).

5.1 Soundness and Exactness Proofs (M_1)

Theorem 5.1 (Soundness). *Let T_A and T_B be two branches with fork point T_0, and suppose every row deleted on a branch is deleted by a statement whose* `deletes_where` *clause the row satisfied at delete time, and that the commit summaries record each deleted row's*

pre-image. If Algorithm 2 returns $V = \emptyset$ for the deltas produced by Algorithm 1 and SemLake commits $T_M = merge(T_0, T_A, T_B)$, then T_M is purge-safe (Definition 3.5) and satisfies Criteria 1–4 across W_1–W_4.

Proof. We verify Conditions 1–4 of Definition 3.5. For Condition 2, Lines 4–5 of Algorithm 2 test p.row_ids $\cap D_X \neq \emptyset$ for all `rows_live` and `foreign_key` predicates on both branches; $V = \emptyset$ guarantees $D_A \cap I = \emptyset$ and $D_B \cap I = \emptyset$. For Condition 3, Lines 11–13 test every conjunct of each `domain`(φ_{dom}) predicate against all `insert` and `update` assignments on the partner branch; $V = \emptyset$ guarantees all assigned column values satisfy φ_{dom}.

For Condition 1, fix `reads_where`(φ_R) $\in \Pi(Y)$ and $r \in D(X)$. By assumption, $\sigma_X(r) \models \varphi_D$ for some `deletes_where`(φ_D) $\in \Pi(X)$. If `WhereOverlap` is true, Lines 9–10 found that neither $\rho_0(r)$ nor $\sigma_X(r)$ satisfies φ_R. Otherwise `WhereOverlap` returned `False`, so some column c admits no value satisfying $T_c(\varphi_D) \wedge T_c(\varphi_R)$ under the per-column test of Section 4.4; hence $\varphi_D \wedge \varphi_R$ is unsatisfiable, so $\sigma_X(r) \not\models \varphi_R$; if $r \in moved_X$, Line 10 found $\rho_0(r) \not\models \varphi_R$, and otherwise $\rho_0(r) = \sigma_X(r)$. For Condition 4, Line 14 flags every changed row that crosses φ_R, changes cols(φ_R) inside it, or cannot be evaluated. Criteria 1, 2, and 4 follow directly from the column-disjoint composition, VSA encapsulation, and $\preceq_{\text{type}}$ schema unification rules in Section 4.5. □

Lemma 5.2 (Exact DAG Replay Since the Fork). *For any branch tip t and fork point $b = LCA(t_{left}, t_{right})$, Algorithm 1 folds every non-merge commit $c \in anc(t) \setminus anc(b)$ once in topological order, excludes every commit in $anc(b)$, and preserves all predicates and open VSA choices across intermediate clean merges.*

Proof. The post-order DFS from t halts along any edge entering *contained* and emits each node $u \notin$ *contained* only after all of its parents outside *contained* have been emitted, yielding anc(t) \ anc(b) in topological order. Crucially, halting on anc(b) prevents dropping valid branch commits in multi-merge chains: if a report branch R (forked from T_0) merges with an update branch U to form M_1, and M_1 later merges with a purge branch P that forked from U, the new LCA is $b = U$. Even though R is not a descendant of U, $R \in$ anc(M_1)\anc(U) is retained and its parent $T_0 \in$ anc(U) maps to *folded*[b] via Known(p), preserving R's read predicate. Meanwhile, for any internal merge node u with fork f, `StripPrefix` removes *folded*[f] before combining its parents, ensuring shared history prefixes are never double-counted as concurrent conflicts. □

5.2 Precision, Complexity, and Scan Overhead Bounds (M_1–M_4)

Precision Boundaries (M_1). Theorem 5.1 and Lemma 5.2 rule out false negatives on W_3, W_4 and false aborts on W_1, but the check can still block a safe merge when: (a) the read clause contains a join or subquery and cannot be evaluated against the deleted rows on its own, (b) a row entered the read range only on branch X before being deleted, which Definition 3.5 conservatively counts, or (c) an update changes a column the reader uses but not its result. Randomized tests check Theorem 5.1: 200 random branch pairs, run directly and again after an intermediate merge, with both merge orders and against a brute-force oracle, find no missed conflict.

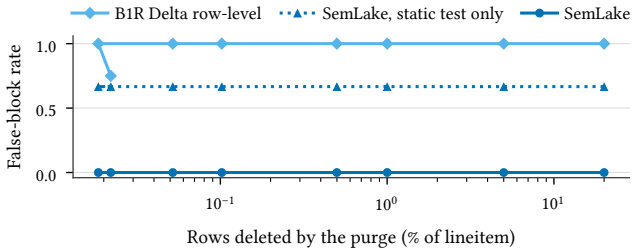


Figure 3: False-block rate vs. delete selectivity ($W_3 \rightarrow M_1$) on TPC-H `lineitem` (SF 0.01, 31 files): a purge whose clause matches none of the rows a concurrent Q6-style report reads (6 of the 31 files). B_1R's file-level read check blocks 0.75 of the merges at the smallest purge and all of them from 0.02% upward. The static test alone blocks the two clause forms outside $\mathcal{F}$ (`NOT`, `OR`; rate 0.67); with the deleted-row check SemLake blocks none.

Merge Time and Cloud S3 API Complexity (M_2, M_3). Let $K = |C(T_0, T_A)| + |C(T_0, T_B)|$ be the number of non-merge commits since T_0, $|\Delta_{\text{rows}}|$ be the number of modified rows, $|\Delta_{\text{files}}|$ be the number of touched files, and $|\Pi|$ be the number of attached predicates. Algorithm 1 issues at most K object-store `GET` requests (M_3), Algorithm 2 and VSA reconciliation run in $O(|\Pi_A| \cdot |\Pi_B| + |\Delta_{\text{rows}}|)$ CPU time (M_2), and physical publication writes 1 delta Parquet file plus 3 small JSON objects (4 `PUT`s), yielding $O(K + |\Delta_{\text{files}}|)$ total S3 API cost and WAF $= 1 + B_{\text{meta}}/B_{\Delta\text{rows}} \approx 1$ (M_3)—independent of total table file count N_{files} and row count N_{rows}.

Post-Merge Analytical Scan Overhead Bound (M_4). Let a table span G_{rg} Parquet row groups, and let $G_{\text{dirty}} \leq G_{\text{rg}}$ be the number of row groups with non-empty Roaring DVs after merging. Because clean row groups (DV $= \emptyset$) execute native vectorized scans at base cost C_{scan} while each dirty row group incurs a relative bitmap-mask factor α, post-merge query latency is bounded by $T_{\text{scan}}(T_M) = C_{\text{scan}} \cdot (1 + \alpha \cdot \frac{G_{\text{dirty}}}{G_{\text{rg}}})$. For localized updates (W_1, where $G_{\text{dirty}} \ll G_{\text{rg}}$), relative scan overhead approaches 0. When cumulative scan overhead across N_{scans} queries exceeds compaction rewrite cost C_{compact} (at crossover point $N^*_{\text{scans}} = \frac{C_{\text{compact}}}{\alpha \cdot C_{\text{scan}} \cdot (G_{\text{dirty}}/G_{\text{rg}})}$), background compaction folds DVs into clean base files and resets $G_{\text{dirty}} \rightarrow 0$.

6 IMPLEMENTATION

We implemented SemLake in Python (about 2,300 lines of engine code in six modules: `metadata.py`, `storage.py`, `merge.py`, `verification.py`, `sql_predicates.py` and `baselines.py`, plus about 5,100 lines of benchmark code).

Data files are encoded as Apache Parquet objects via `pyarrow`, with a hidden `__row_id` column tracking Iceberg v3 row lineage [5]. Positional deletion vectors are implemented using `pyroaring` (`BitMap.serialize` and bitwise union `|`). Analytical queries in metric M_4 execute in DuckDB over Parquet files on local storage; the read experiments store a DV as a Parquet mask of booleans and apply it with a positional join, while clean row groups are read without a mask. To support deep commit histories and high-concurrency branching with minimal memory footprint, snapshot row states use a copy-on-write structural overlay (`RowView`): each commit stores only modified records and deleted row IDs, walking the parent

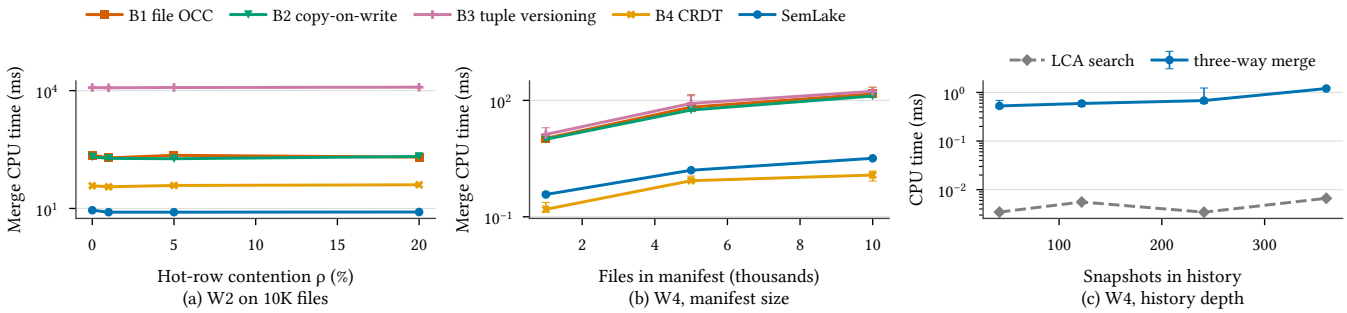


Figure 4: Three-way merge CPU latency scalability (M_2) across three lakehouse dimensions ordered by workload ($W_2 \rightarrow W_4$): (a) hotspot row contention ρ from 0% to 20% on a 10K-file table (W_2), (b) manifest depth from 1K to 10K files (W_4), and (c) commit DAG history depth from 42 to 360 snapshots (W_4). Median of 5 runs; bars show the range.

chain on lookup without copying unchanged rows. In the commit graph, each snapshot caches its ancestor-generation map upon creation, allowing `lca` to run a generation-ordered max-heap search that terminates as soon as the highest open generation falls below the best common ancestor found.

To profile cloud storage costs deterministically (M_3), our in-memory object store logs exact counts of `PUT`, `GET`, and `LIST` calls and transferred bytes, applying public AWS S3 Standard (`us-east-1`) pricing ($0.005 per 1,000 `PUT`/`LIST` calls, $0.0004 per 1,000 `GET` calls [3]) and the analytical cloud latency model $T_{\text{est}} = T_{\text{cpu}} + N_{\text{put}} \cdot 100\,\text{ms} + N_{\text{get}} \cdot 30\,\text{ms} + N_{\text{list}} \cdot 50\,\text{ms} + B_{\text{put}}/(100\,\text{MB/s})$. All reported CPU latencies are real wall-clock measurements without artificial `sleep()` calls; no experiment runs against S3. B_1, B_1R, B_2 and B_6 are rule models evaluated on the same snapshots as SemLake: they compute what each system's documented rule decides, and a test confirms B_1's rule against a real `delta-rs` table. B_3 and B_4 are our reimplementations, and B_5 is the Dolt binary driven through its SQL interface with `dolt merge`. Compaction and the group-commit service are implemented in the benchmark harness.

7 EXPERIMENTAL EVALUATION

We evaluate semantic correctness (M_1, §7.2), merge latency (M_2, §7.3), object-store cost (M_3, §7.4), post-merge scan overhead (M_4, §7.5), ablations (§7.6) and the swarm extension (§7.7).

7.1 Experimental Setup and Baselines

All experiments execute on one machine with 4 cores, Python 3.13.16, DuckDB 1.5.6, `pyarrow` 26.0.0, `pyroaring`, `sqlglot`, and Dolt. TPC-H data is real `lineitem` generated with `tpchgen-cli`, a port of the TPC-H generator [33]. Timed merges run once untimed and then 5 times, and we report the median with the range; DuckDB scans run in separate processes; the swarm runs 5 seeds and reports means with 95% confidence intervals. Following the Branch-Execute-Merge protocol adapted from LST-Bench [10], we evaluate all workloads in canonical order: W_1 (shared-file disjoint updates), W_2 (hotspot key contention $\rho \in [0\%, 20\%]$), W_3 (cross-branch predicate conflicts over a 34-case labelled semantic suite spanning W_1, W_3, W_4 and 42 TPC-H [33] refresh-versus-query branch pairs), W_4 (schema evolution and manifest/DAG scaling from 1K to 10K files and 42 to 360 snapshots), and the extension workload W_{swarm} (8 to 64 concurrent agents combining W_1–W_3).

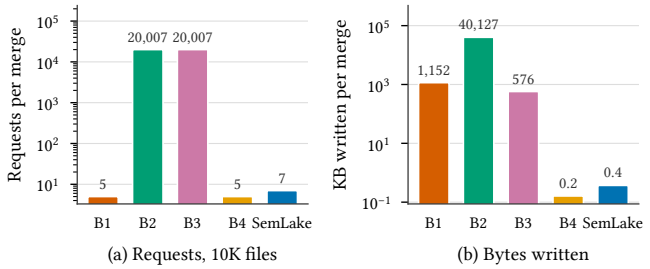
Table 3: Conflict detection on (a) the labelled scenarios and (b) the TPC-H refresh-versus-report pairs. TP and FP count reported semantic conflicts; Ab. counts write-conflict aborts, which block a merge without naming a semantic reason.

	(a) 34 scenarios, 18 conflicts					(b) 42 TPC-H pairs, 22 conflicts			
Rule	TP	FP	Ab.	Rec.	Prec.	TP	FP	Rec.	Prec.
B1 file OCC	0	0	1	0.00	–	0	0	0.00	–
B1R Delta-style	14	8	0	0.78	0.64	22	3	1.00	0.88
B2 copy-on-write	0	0	0	0.00	–	0	0	0.00	–
B3 tuple versioning	0	0	0	0.00	–	0	0	0.00	–
B4 CRDT	0	0	0	0.00	–	0	0	0.00	–
B5 Dolt	2	0	0	0.11	1.00	0	0	0.00	–
B6 SSI	0	0	0	0.00	–	0	0	0.00	–
SemLake, static test only	18	2	0	1.00	0.90	22	8	1.00	0.73
SemLake	18	0	0	1.00	1.00	22	0	1.00	1.00

Following Table 1, we compare SemLake against all six baseline families in canonical order (B_1, B_1R, B_2, B_3, B_4, B_5, B_6): B_1 (File-level OCC [6]), B_1**R** (Delta row-write variant of B_1 with file-level read conflict checks [24]), B_2 (Copy-on-Write row merge [32]), B_3 (Relational tuple versioning [22, 26]), B_4 (Automatic CRDT views [17]), B_5 (Constraint-aware cell merge, the Dolt binary executing native `dolt merge` with SQL `FOREIGN KEY` and `CHECK` constraints [14]), and B_6 (Classical SSI [9], detecting two-edge *rw*-antidependency cycles). All baselines are scored on M_1 (Table 3); the storage benchmarks (M_2–M_4) cover B_1, B_1R and B_2–B_4, because B_5 keeps its own Prolly-tree store and B_6 uses OLTP `SIREAD` locks without Parquet manifests.

7.2 Semantic Merge Correctness (M_1)

Table 3 evaluates conflict recall and precision (M_1) across all baselines (B_1–B_6 and B_1R) on two benchmarks: (a) the 34 labelled merge scenarios spanning W_1 (shared-file disjoint updates), W_3 (equality, range, `BETWEEN`, `IN`, multi-column `AND`, boundary ≤ 3 vs. ≥ 3, $\neq$, unparsed `NOT`/`OR`, domain invariants, foreign keys, `rows_live`, and updates and inserts across or within a read range), and W_4 (additive schema evolution); and (b) 42 TPC-H [33] branch pairs (W_3) on real `lineitem` at scale factor 0.01 (60,175 rows sorted by `l_shipdate` in 31 files), pitting six refresh branches (four ship-date purges and two RF2-style deletes of order keys) against seven report branches that read with the `lineitem` filters of Q1, Q3, Q6, Q7, Q10, Q12 and Q14. A pair is a true conflict if a deleted row satisfies the report's filter; 22 pairs are.



(a) Requests, 10K files (b) Bytes written

Figure 5: Object-store request count and payload bytes written (M_3) on workload W_1 at 10,000 files (B_1–B_4 vs. SemLake). SemLake issues 7 requests vs. 20,007 for Copy-on-Write (B_2) and Tuple Versioning (B_3).

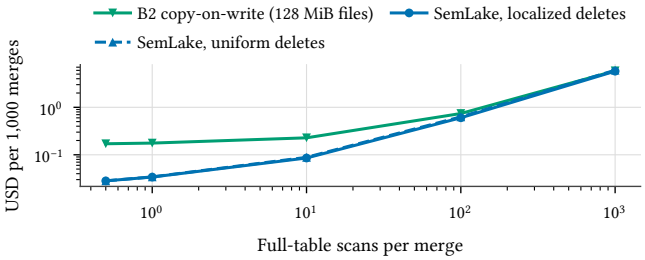


Figure 6: Total dollar cost per 1,000 merged changes (M_3, M_4: S3 API requests + 1-day storage + merge and scan CPU) against downstream full-table scans per merge, for 128 MiB base files.

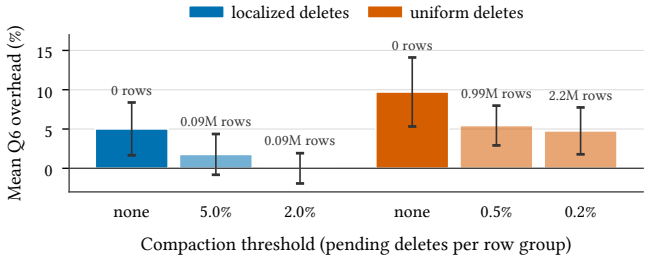


Figure 7: Mean TPC-H Q6 overhead (M_4, SF 1) over 40 merges that each delete 0.1% of the rows, by compaction threshold (fraction of a row group's rows with pending deletes). Bars show 95% intervals over the merges; labels give rows rewritten per merge.

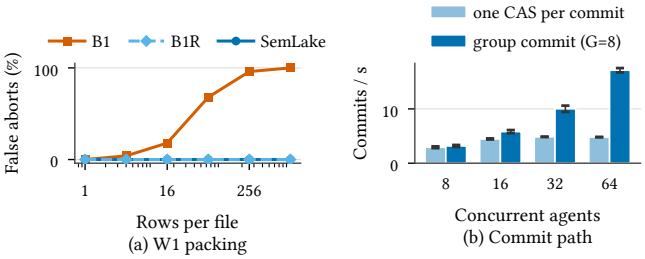


Figure 8: Ablation studies ($W_1 \rightarrow W_{\text{swarm}}$): (a) false-conflict abort rate (M_1) of B_1 vs. SemLake on W_1 as rows per Parquet file increase from 1 to 1,024; (b) applied commits per second on W_{swarm} (M_2) with one catalog CAS per commit vs. group commit ($G = 8$), 8 to 64 agents.

Examining the systems in canonical order: B_1 (File OCC) tracks only physical file writes and scores 0.00 recall across both suites while suffering false aborts on W_1. B_1R (Delta-style file-read tracking) catches the read-delete hazards whose rows share a file with the deleted rows (recall 0.78 on the 34-case suite, missing domain/FK invariants; recall 1.00 on TPC-H), but blocks 8 safe merges on the 34-case suite (precision 0.64) and 3 safe TPC-H pairs (precision 0.88) whose purges delete rows in files the report scanned. Row/tuple/CRDT baselines B_2, B_3, and B_4 lack predicate verification and score 0.00 recall on both benchmarks. B_5 (Dolt) achieves recall 0.11 (2/18) on the 34-case suite and 0.00 on TPC-H: it catches the declared `FOREIGN KEY` and `CHECK` cases, but a report's filter cannot be declared as a table constraint (W_3), and Dolt refuses to add a `CHECK` constraint that legacy rows in T_0 violate. Confirming Proposition 3.6, B_6 (Classical SSI) also scores 0.00 recall across both benchmarks: because a report branch T_B and a purge branch T_A induce only a single one-way dependency $T_B \xrightarrow{rw} T_A$ (serializable as $T_B \prec T_A$) without a back-edge $T_A \xrightarrow{rw} T_B$, SSI's dangerous-structure detector never triggers. By contrast, SemLake achieves **1.00 recall** (18/18) and **1.00 precision** on the 34-case suite, and **1.00 recall** (22/22) with **1.00 precision** on TPC-H; with the deleted-row check disabled, precision falls to 0.90 and 0.73.

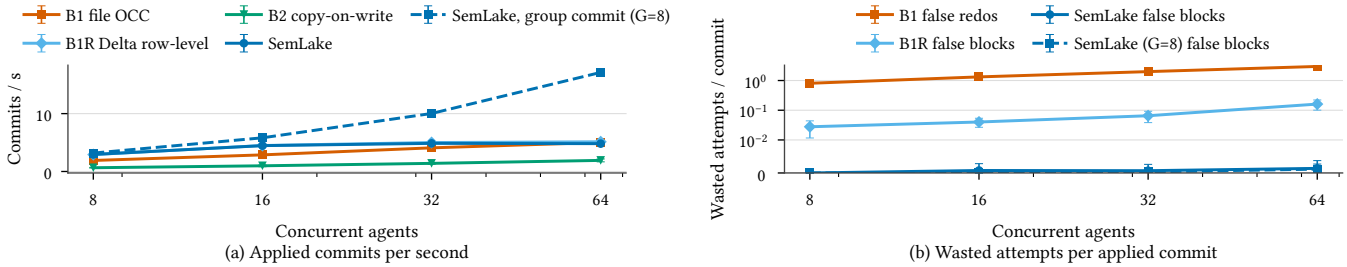
5 refreshes that only update or insert rows (a discount fix, a reshipment, returns and new quantities on 40 orders, and an RF1-style insert of 15 orders) give 35 more pairs; 10 are true conflicts, where the report's value changes. SemLake catches 10 (recall 1.00, precision 1.00); without Line 14 it catches 0. B_1R reaches recall 0.50 at precision 0.24: it blocks a report that scanned a rewritten file whether or not its value changed, and misses inserts, which land in new files. B_1, B_2–B_4 and B_6 catch 0. B_5 is omitted: its first merge here had not finished after 12 minutes.

Figure 3 further isolates the impact of delete selectivity (W_3) on false blocks. A report reads with Q6's filter, which touches 6 of the 31 files, and a purge deletes a random range of part keys restricted to discounts Q6 does not read, so no pair is a true conflict; each point is 12 purges in three clause forms (conjunctive, `NOT` and `OR`), deleting from 0.019% to 20% of the table. B_1R blocks the report as soon as the purge touches one of the files Q6 reads: its false-block rate is 1.00 at 0.1% of rows. Static-only SemLake blocks the `NOT` and `OR` forms (rate 0.67), whereas full SemLake has a **0.00 false-block rate** at every selectivity.

Key Insight (M_1): *Predicate-level verification over commit summaries eliminates silent read-delete, read-update and domain corruption (W_3) with 1.00 recall, avoiding both the low recall of B_1, B_2–B_6 and the false blocks of file-granularity read tracking (B_1R).*

7.3 Merge Latency and Metadata Scalability (M_2)

Figure 4 evaluates three-way merge CPU latency (M_2) across three scaling dimensions ordered by workload ($W_2 \rightarrow W_4$): (1) **Hotspot row contention (W_2, Figure 4a):** on a 10,000-file table, SemLake merges in 9.02 ms at $\rho = 0$ and 8.12 ms at $\rho = 20\%$, where colliding cells become VSA choice nodes; B_1 takes 227.3 ms and 203.2 ms because it compares full file lists. B_4 takes 37.9 ms at $\rho = 0$, but it checks nothing: it overwrites one side of every collision. (2) **Manifest depth (W_4, Figure 4b):** as the manifest grows from 1K to 10K Parquet files, SemLake merges a schema change and an append in



(a) Applied commits per second

(b) Wasted attempts per applied commit

Figure 9: Extension case study on agent swarms (W_{swarm}): (a) applied commits per second (M_2) from 8 to 64 concurrent agents (B_1, B_1R, B_2 vs. SemLake; with a 10 ms catalog round trip, one CAS per commit levels off near 5 commits/s, while SemLake with group commit ($G = 8$) reaches 17.1 commits/s at 64 agents; 5 seeds, 95% intervals); (b) wasted agent work (M_1: false transaction redos in B_1 and false read blocks in B_1R) per applied commit.

0.38 ms and 3.22 ms, while B_1, B_2 and B_3 take 147.0 ms, 129.8 ms and 171.4 ms at 10K files, and B_3 issues 20,009 object-store calls to rebuild its dictionaries. (3) **Commit DAG history depth (W_4, Figure 4c):** from 42 to 360 snapshots, generation-pruned LCA discovery takes 3.5 and 6.6 μs and the merge 0.53 ms and 1.20 ms. Each merge replays one commit per side, so replay does not explain the growth; we did not isolate its cause. CPU time is not the whole story: under the latency model of Section 6, a W_2 merge at $\rho = 0$ is estimated at 0.55 s for SemLake and 0.48 s for B_1, because SemLake reads one summary per commit and writes more metadata objects. **Key Insight (M_2):** *By isolating hotspot cell collisions in VSA choice nodes (W_2) and inheriting unchanged file descriptors by reference (W_4), SemLake prevents contention-induced abort storms and keeps merge CPU time 40–50× below the file-level baselines at 10K files, with a request count independent of N_{files}.*

7.4 Cloud Object-Store Cost Efficiency (M_3)

Figure 5 evaluates S3 request counts and write amplification (M_3) on W_1 across a 10,000-file table where two branches update disjoint rows across 5,000 files each. SemLake completes the merge in **7 S3 requests** (3 `PUT`, 3 `GET`, 1 `LIST`), writing 380 bytes of metadata ($0.0000212) and rewriting no base file. Copy-on-Write B_2 reads and rewrites all 10,000 modified files, issuing 20,007 S3 requests (10,001 `PUT`, 10,005 `GET`, 1 `LIST`) at $0.054012—a **99.96% S3 request-cost reduction** for SemLake. B_1 issues only 5 requests, but it aborts this merge. Similarly, on W_4, SemLake issues 7 S3 requests ($O(\Delta_{\text{manifest}})$) compared to 20,009 requests in B_3.

While keeping deletion vectors uncompacted minimizes merge-time write I/O (M_3), it introduces bitmap filtering work during subsequent analytical reads (M_4). Validating the crossover bound N^*_{scans} from Section 5.2, Figure 6 models total dollar cost per 1,000 merges (combining the measured S3 API requests, one day of S3 storage for written bytes, 128 MiB Parquet file rewrites for B_2, and CPU at $0.10/hour, using the scan overheads of Table 4) against downstream full-table scans per merge: (1) at 10 scans per merge, 1,000 merges with open DVs cost $0.086 in SemLake for localized deletes ($0.088 for uniform deletes) vs. $0.228 for CoW B_2; (2) leaving DVs open stays cheaper than rewriting until **3,094 scans per merge** when deletes are localized to a single row group (W_1), and **446 scans per merge** when uniform deletes dirty every row group. Past that point the merged files should be compacted (Section 7.5).

Table 4: TPC-H Q6 on SF 1 `lineitem` after a merge left deletes in a DV: median time of 5 processes, and overhead with a 95% confidence interval.

Deletes	Rows	Clean groups	Base (ms)	DV (ms)	Overhead (95% CI)
localized	1%	60/61	190.5	198.2	+0.9% (−3.4 to +6.9%)
uniform	0.1%	0/61	194.0	206.8	+4.2% (−1.7 to +12.5%)
uniform	1%	0/61	187.4	207.2	+6.0% (−5.3 to +23.1%)
uniform	5%	0/61	178.6	187.8	+5.4% (+0.2 to +14.2%)

Key Insight (M_3): *Compiling branch merges into Roaring DV unions and delta manifests cuts object-store requests by about three orders of magnitude, remaining more cost-effective than eager CoW rewrites (B_2) for up to 446–3,094 downstream full-table scans per merge.*

7.5 Post-Merge Analytical Scan Overhead (M_4)

Table 4 reports the post-merge read penalty (M_4) when running TPC-H Q6 [33] in DuckDB (one thread) over real SF 1 `lineitem` (6,001,215 rows in 61 row groups of 100,000 rows) after a merge left a DV open. Clean row groups with empty deletion vectors ($G_{\text{rg}} - G_{\text{dirty}}$) are scanned directly from untouched base files; dirty row groups (G_{dirty}) apply a positional boolean mask join.

Under localized updates (W_1, where 1% of the rows are deleted inside one row group, $G_{\text{dirty}}/G_{\text{rg}} = 1/61$), the median scan overhead is **+0.9%** (95% CI: −3.4% to +6.9%, within our < 8% target). When the same deletes are spread over every row group ($G_{\text{dirty}}/G_{\text{rg}} = 1$), every group needs the mask, and the overhead is +6.0%. Both intervals are wide: the masking cost is small next to the scan itself. At SF 10 (59,986,052 rows) the overheads are −1.4% (−5.6% to +2.7%) localized and +5.6% (+2.9% to +9.0%) uniform. We did not measure a native DV reader.

Figure 7 follows a table through 40 merges, each deleting 0.1% of its rows, and measures Q6 after every merge; a policy rewrites a row group once its pending deletes pass a threshold. Without compaction, the mean overhead is 5.0% for localized deletes and 9.7% for uniform ones. With localized deletes, a 2% threshold brings it to 0.0% and rewrites 89,549 rows per merge, almost one row group. With uniform deletes, the best threshold we tried (0.2%) gives 4.8% but rewrites 2,233,892 rows per merge, over a third of the table, because every row group crosses the threshold at about the same time. Compaction therefore pays off for localized deletes; for uniform deletes it costs nearly a copy-on-write rewrite to remove a few percent of scan time.

Table 5: Simulated swarm, 60 s per run, mean of 5 seeds (± 95% CI). Haz.: hazards committed and caught, summed over seeds. Blocks and redos: false semantic blocks and false file-conflict redos per run.

N	Rule	Commits/s	Haz.	Caught	Blocks	Redos
8	B1 file OCC	1.90 ± 0.21	2	0	0.0	91.4
8	B1R Delta-style	2.92 ± 0.19	2	2	4.8	0.0
8	B2 copy-on-write	0.65 ± 0.06	0	0	0.0	0.0
8	SemLake	2.94 ± 0.16	0	4	0.0	0.0
8	SemLake, group commit	3.18 ± 0.18	0	4	0.0	0.0
32	B1 file OCC	4.09 ± 0.11	5	0	0.0	488.8
32	B1R Delta-style	5.00 ± 0.04	4	9	19.6	0.0
32	B2 copy-on-write	1.40 ± 0.15	0	0	0.0	0.0
32	SemLake	4.86 ± 0.04	0	12	0.2	0.0
32	SemLake, group commit	10.02 ± 0.58	0	19	0.2	0.0
64	B1 file OCC	5.02 ± 0.05	20	0	0.0	895.6
64	B1R Delta-style	5.13 ± 0.01	11	20	49.6	0.0
64	B2 copy-on-write	1.92 ± 0.25	0	0	0.0	0.0
64	SemLake	4.81 ± 0.01	0	43	0.4	0.0
64	SemLake, group commit	17.12 ± 0.40	0	59	1.2	0.0

Key Insight (M_4): *Skipping bitmap filtering on clean row groups keeps post-merge analytical scan overhead negligible (+0.9% at SF 1) for partitioned or localized branch updates (W_1).*

7.6 Ablation Studies

We isolate five core design choices in SemLake ordered by workload ($W_1 \rightarrow W_3 \rightarrow W_4 \rightarrow W_{\text{swarm}}$): (1) **Parquet file packing density ($W_1 \rightarrow M_1$, Figure 8a):** two branches each update 4 random rows of a 1,024-row table; as rows packed per Parquet file increase from 1 to 1,024, B_1's false-abort rate climbs from 0 to 1.00, whereas B_1R and SemLake maintain a **0% false-abort rate** regardless of file size because both compare rows. (2) **Tier-2 deleted-row check ($W_3 \rightarrow M_1$, Figure 3):** disabling `AnyMatch` in Algorithm 2 leaves recall unchanged but lowers precision from 1.00 to 0.90 on the 34-case suite and from 1.00 to 0.73 on the TPC-H pairs, and raises the false-block rate for the `NOT` and `OR` forms in Figure 3 from 0.00 to 0.67. (2b) **Update check ($W_3 \rightarrow M_1$):** without Line 14, recall falls to 0.78 on the suite and 0.00 on the update pairs; delete pairs are unchanged. (3) **Exact DAG replay since the fork ($W_4 \rightarrow M_1$):** tip-only replay loses the predicates of earlier merges and lets through a report merged once before it meets a purge, which Algorithm 1 blocks. (4) **Catalog commit-path batching ($W_{\text{swarm}} \rightarrow M_2$, Figure 8b):** with a 10 ms catalog round trip, one CAS per commit reaches 4.8 commits/s at 64 agents, whereas group commit ($G = 8$) reaches **17.1 commits/s**.

7.7 Extension Case Study: Concurrent Agent Swarms (W_{swarm})

Finally, we evaluate SemLake on the agent-swarm extension W_{swarm} [4, 20, 25, 36] across M_1–M_3 in a discrete-event simulation. N agents repeatedly branch from `main`, work for a random time (mean 2 s), and commit one task: an update of four rows (80%), a range delete (5%), or a report that reads a range and writes an aggregate (15%), on a table of 32 files of 32 rows. Publishing is a CAS with a 10 ms catalog round trip. SemLake runs on the real engine, and each merge advances simulated time by its measured CPU time plus its counted requests under the latency model; the baselines are rule models charged with the same model. A report and a delete that ran concurrently over shared rows are a hazard if both commit. Each run lasts 60 s; we also run a purge-heavy mix (15% deletes, 30% reports) at 8 and 32 agents, and hazard totals cover both mixes. Table 5 compares SemLake against lakehouse catalog CAS baselines ($B_1 \rightarrow B_1\text{R} \rightarrow B_2$; B_3–B_5 lack manifest CAS and B_6 lacks branched snapshots).

B_1 redoes work on every file overlap and B_2 rewrites every touched file. B_1R and SemLake both avoid file conflicts, but B_1R blocks a report whenever a concurrent delete touched a file it read: 19.6 blocks per run at 32 agents, against 0.2 for SemLake. Over all runs, B_1 committed 63 hazards, B_2 9, B_1R 27, and SemLake **0**. B_1R still commits hazards because its read check is one-directional: a delete that commits after a report read nothing the report wrote, so it goes through. Figure 9 shows throughput (M_2) and wasted work (M_1) up to 64 agents. With one CAS per commit, SemLake's throughput is limited by the merge's object-store round trips (4.8 commits/s at 64 agents); group commit ($G = 8$) raises it to **17.1 commits/s** with **zero committed hazards**.

8 RELATED WORK

Lakehouse Storage and Row-Level Operations. Delta Lake [6], the Lakehouse paradigm [35], and comparative storage studies [10, 21, 23] established metadata-driven ACID transactions over cloud object stores. Okolnychyi et al. [28], Apache Iceberg v3 [5], and Delta Lake deletion vectors [24] introduced Roaring DVs and row lineage for single-branch MoR updates, while Apache Hudi [31] and Lance [11] provide multi-modal metadata and point-lookup indices. Durner et al. [15] quantified the S3 request-cost model motivating our $O(\Delta_{\text{manifest}})$ design. LakeVilla [19], Götz et al. [18], and TreeCat [27] coordinate multi-table commits and catalog trees, but omit cross-branch predicate conflict verification.

Dataset Versioning and Branching. Dataset versioning principles [8], DataHub [7], Decibel [26], OrpheusDB [22], and Fork-Base [34] pioneered storage layouts for branched datasets. Bauplan [32] provides zero-copy branch references for cloud WAP pipelines, and Dolt [14] performs cell-level merges over Prolly trees with SQL schema constraints, missing query read predicates and partial-domain invariants.

Program Merge and Version-Space Algebra. Sousa et al. [30] formalized verified three-way program merge over ASTs, Zhu and He [38] introduced VSA to represent conflicting AST candidates, and Zhu et al. [37] combined top-down and bottom-up traversals. Subsequent work detects behavioral conflicts via unit tests [12] or static analysis [13]. SemLake brings verified three-way merge and VSA from ASTs to cloud lakehouses by lifting verification to SQL commit predicates and Roaring DVs.

Concurrency Theory and Replicated Views. Eswaran et al. [16] introduced predicate locks, Adya [1] formalized multi-version dependency graphs, and Cahill et al. [9] developed Serializable Snapshot Isolation (SSI). Proposition 3.6 shows that purge-safe branch merging and two-branch serializability are incomparable, because both branches' outputs coexist in T_M. CRDTs [2] and CRDV [17] guarantee automatic state convergence without branch isolation or semantic conflict blocking.

Extensions to Agentic Systems. Git4Data [20] and Chronos [36] provide copy-on-write and bolt-on branching for AI agents, BranchBench [4] evaluates branching vs. query latency under agentic workloads, and ATR [25] studies selective revalidation for long-running agents. SemLake adds the missing three-way semantic merge to these substrates.

9 LIMITATIONS AND CONCLUSION

Our evaluation highlights four limitations pointing to future work: (1) *Direct cloud integration*: Parquet I/O, bitmaps, DuckDB scans, and Dolt runs are physical, but the object store is in memory and S3 costs and latencies are modelled; B_1, B_1R, B_2 and B_6 are rule models, and the swarm is a simulation. Native AWS S3 and Iceberg v3 Puffin integration is planned. (2) *Predicate scope*: Condition 4 compares rows rather than query results, so it blocks an update that changes a column a report uses without changing the report's value, and a read clause with a join or subquery cannot be checked against rows and is treated as overlapping. (3) *Scale and scans*: the TPC-H pairs use scale factor 0.01, the scans scale factors 1 and 10 on one thread; DVs are applied through a Parquet mask and a positional join rather than a native bitmap reader, and compaction lives in the benchmark harness. (4) *Multi-table merges*: extending group commit to multi-table lakehouse transactions [18, 19] remains open.

SemLake brings verified three-way semantic merge to branched data lakehouses. By decoupling predicate-level conflict verification over compact commit summaries from zero-copy physical reconciliation over Roaring deletion vectors, SemLake eliminates silent semantic corruption while scaling cloud API cost solely with branch delta size.

REFERENCES

[1] Atul Adya. 1999. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions.* Ph.D. Dissertation (Tech. Rep. MIT-LCS-TR-786). Massachusetts Institute of Technology.

[2] Paulo Sérgio Almeida. 2024. Approaches to Conflict-free Replicated Data Types. *ACM Comput. Surv.* 56, 8 (2024), 202:1–202:36.

[3] Amazon Web Services. 2026. *Amazon S3 Pricing: S3 Standard, US East (N. Virginia).* Price List. Amazon Web Services, Inc. https://aws.amazon.com/s3/pricing/.

[4] Elaine Ang, In Keun Kim, Sam Weldon, Kevin Durand, Kostis Kaffes, and Eugene Wu. 2026. BranchBench: Aligning Database Branching with Agentic Demands. *arXiv preprint arXiv:2604.17180* (2026).

[5] Apache Software Foundation. 2025. *Apache Iceberg Table Format Specification: Version 3 (Deletion Vectors and Row Lineage).* Technical Specification Format Version 3. The Apache Software Foundation. https://iceberg.apache.org/spec/.

[6] Michael Armbrust, Tathagata Das, Liwen Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proc. VLDB Endow.* 13, 12 (2020), 3411–3424.

[7] Anant Bhardwaj, Souvik Bhattacherjee, Amit Chavan, Amol Deshpande, Aaron J. Elmore, Samuel Madden, and Aditya Parameswaran. 2015. DataHub: Collaborative Data Science and Dataset Version Management at Scale. In *Proc. Conf. on Innovative Data Systems Research (CIDR).*

[8] Souvik Bhattacherjee, Amit Chavan, Silu Huang, Amol Deshpande, and Aditya Parameswaran. 2015. Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff. *Proc. VLDB Endow.* 8, 12 (2015), 1346–1357.

[9] Michael J. Cahill, Uwe Röhm, and Alan D. Fekete. 2009. Serializable Isolation for Snapshot Databases. *ACM Trans. Database Syst.* 34, 4 (2009), 20:1–20:42.

[10] Jesús Camacho-Rodríguez, Ashvin Agrawal, Anja Gruenheid, Ashit Gosalia, Cristian Petculescu, Josep Aguilar-Saborit, Avrilia Floratou, Carlo Curino, and Raghu Ramakrishnan. 2024. LST-Bench: Benchmarking Log-Structured Tables in the Cloud. *Proc. ACM Manag. Data* 2, 1 (2024), 56:1–56:26.

[11] Chang She Chang, Weston Pace, Will Kunen, et al. 2025. Lance: Efficient Random Access in Columnar Storage through Adaptive Structural Encodings. *arXiv preprint arXiv:2504.15247* (2025).

[12] Leuson da Silva, Paulo Borba, Arthur Toni, and Rodrigo Bonifácio. 2024. Detecting Semantic Conflicts with Unit Tests. *Journal of Systems and Software* 212 (2024), 112006.

[13] Galileu Santos de Jesus, Paulo Borba, Rodrigo Bonifácio, and Matheus Barbosa de Oliveira. 2024. Lightweight Semantic Conflict Detection with Static Analysis. In *Proc. IEEE/ACM 46th Int. Conf. on Software Engineering: Companion Proceedings (ICSE-Companion '24).* 272–276.

[14] DoltHub, Inc. 2026. *Dolt: Git for Data (Version-Controlled SQL Database).* Software Repository Version 2.5.0, built from source. DoltHub, Inc. https://github.com/dolthub/dolt.

[15] Dominik Durner, Viktor Leis, and Thomas Neumann. 2023. Exploiting Cloud Object Storage for High-Performance Analytics. *Proc. VLDB Endow.* 16, 11 (2023), 2769–2782.

[16] Kapali P. Eswaran, Jim N. Gray, Raymond A. Lorie, and Irving L. Traiger. 1976. The Notions of Consistency and Predicate Locks in a Database System. *Commun. ACM* 19, 11 (1976), 624–633.

[17] Nuno Faria and José Pereira. 2025. CRDV: Conflict-free Replicated Data Views. *Proc. ACM Manag. Data* 3, 1 (2025), 42:1–42:25.

[18] Tobias Götz, Daniel Ritter, Muhammad El-Hindi, and Jana Giceva. 2026. Interoperable ACID Transactions for Open Table Formats. *Proc. VLDB Endow.* 19, 11 (2026), 3216–3230.

[19] Tobias Götz, Daniel Ritter, and Jana Giceva. 2025. LakeVilla: A Modular and Non-Invasive Toolbox for Lakehouse Transactions. *arXiv preprint arXiv:2504.20768* (2025).

[20] Hongshen Gou, Zuyu Zhang, Yuze Sun, Peng Xu, Feng Tian, Long Wang, and Jianguo Wang. 2026. Git4Data: Database-Native Version Control for AI Agents. *arXiv preprint arXiv:2609.02106* (2026).

[21] Ahmed A. Harby and Farhana Zulkernine. 2025. Data Lakehouse: A Survey and Experimental Study. *Information Systems* 127 (2025), 102460.

[22] Silu Huang, Liqi Xu, Mengxiao Xia, and Aditya Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. *Proc. VLDB Endow.* 10, 10 (2017), 1130–1141.

[23] Paras Jain, Peter Kraft, Conor Power, Tathagata Das, Ion Stoica, and Matei Zaharia. 2023. Analyzing and Comparing Lakehouse Storage Systems. In *Proc. Conf. on Innovative Data Systems Research (CIDR).*

[24] Linux Foundation Delta Lake Project. 2024. *Delta Transaction Log Protocol: Deletion Vectors and Row Tracking Specification.* Protocol Specification Reader v3 / Writer v7. The Linux Foundation. https://github.com/delta-io/delta/blob/master/PROTOCOL.md.

[25] Yongjian Lyu, Yang Ren, Ruofei Lai, and Wenting Liu. 2026. From Version Conflicts to Decision Conflicts: Selective Revalidation for Long-Running AI Agents. *arXiv preprint arXiv:2609.08015* (2026).

[26] Michael Maddox, David Goehring, Aaron J. Elmore, Samuel Madden, Aditya Parameswaran, and Amol Deshpande. 2016. Decibel: The Relational Dataset Branching System. *Proc. VLDB Endow.* 9, 9 (2016), 624–635.

[27] Seongjin Oh, Hoang Nguyen, and Yanlei Diao. 2025. TreeCat: Standalone Catalog Engine for Large Data Systems. *Proc. VLDB Endow.* 18, 11 (2025), 3845–3858.

[28] Anton Okolnychyi, Ryan Blue, Russell Spitzer, Jack Chen, and John Wang. 2024. Petabyte-Scale Row-Level Operations in Data Lakehouses. *Proc. VLDB Endow.* 17, 12 (2024), 4159–4171.

[29] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19).* ACM, Amsterdam, The Netherlands, 1981–1984.

[30] Marcelo Sousa, Isil Dillig, and Shuvendu K. Lahiri. 2018. Verified Three-Way Program Merge. *Proc. ACM Program. Lang.* 2, OOPSLA (2018), 165:1–165:29.

[31] Sagar Sumit, Prashant Wason, and Sivabalan Narayanan. 2026. Metadata-as-Data in Apache Hudi: A Multi-Modal Index Substrate for Lakehouse Tables. In *1st International Workshop on Data FORMATS for Modern Architectures and Workloads (FORMATS '26).* ACM, Bengaluru, India, 5.

[32] Jacopo Tagliabue, Luca Bigon, Ciro Greco, et al. 2024. Bauplan: Zero-Copy, Scale-Up FaaS for Data Pipelines. In *Proc. 10th Int. Workshop on Serverless Computing (WoSC '24).* 31–36.

[33] Transaction Processing Performance Council. 2022. *TPC Benchmark H (TPC-H) Standard Specification.* Standard Specification Revision 3.0.1. Transaction Processing Performance Council (TPC). http://www.tpc.org/tpch/.

[34] Sheng Wang, Tien Tuan Anh Dinh, Qian Lin, Zhongle Xie, Meihui Chang, Gang Chen, and Beng Chin Ooi. 2018. ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications. *Proc. VLDB Endow.* 11, 10 (2018), 1137–1150.

[35] Matei Zaharia, Ali Ghodsi, Reynold Xin, and Michael Armbrust. 2021. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Proc. Conf. on Innovative Data Systems Research (CIDR).*

[36] Xinjing Zhou, Jason Mohoney, Samuel Madden, Michael Stonebraker, and Lei Cao. 2026. Chronos: Efficient Bolt-On Branching Across Data Stores for Stateful Agentic Applications. *arXiv preprint arXiv:2609.14889* (2026).

[37] Fengmin Zhu, Xingyu Xie, Dongyu Feng, Na Meng, and Fei He. 2024. On the Methodology of Three-Way Structured Merge in Version Control Systems: Top-Down, Bottom-Up, or Both. *Journal of Systems Architecture* 152 (2024), 103156.

[38] He Zhu and Fei He. 2018. Conflict Resolution for Structured Merge via Version Space Algebra. *Proc. ACM Program. Lang.* 2, OOPSLA (2018), 166:1–166:28.